

DE Bahn

End-to-end pipeline construction

Changes to German Train schedules analysis



[Snehith Madhusoodhanan](#), [DE Bahn GitHub Repo](#)

Introduction

This is a report on how I built an end-to-end pipeline to ingest, store, transform and serve the changes made to the planned time table of German trains. DB (Deutsch Bahn) provides a public API called Timetables through [DB API Marketplace](#). Timetables API provides 3 endpoints to obtain planned timetable (1) of a particular day, full changes (2) that are planned for a particular day, recent changes (3) that stores the new changes for a time duration of two minutes and station metadata (4). Since there are over 10K train stations in DE, I have focussed only on the existing 150 central stations.

I chose [Databricks Free edition](#) as my data platform. The ingestion pipelines and transformation pipelines run using the Serverless SQL Warehouse provided by Databricks.

Ingestion

I ingested all the station's metadata like EVA number, which is a unique number assigned to each station in Germany, through the station API endpoint. Everyday, at 6am, the full changes that are already decided for the day are ingested through the full changes endpoint. From here on, the changes that are made to the train schedule are retrieved through the recent changes endpoint, by calling this endpoint every two minutes for each of the central stations, starting from 06:02. Ingestion pipelines for collecting the station metadata and full changes run as Databricks jobs, but for collecting the recent changes,

scheduling a Databricks job is not sufficient as I hit the daily compute quota after a few hours. The workaround to collecting the recent changes can be found in the [Orchestration](#).

Sometimes, I found duplicates in the ingested data. So, I deduped them using a row-number based approach.

Storage

I have chosen the AWS (Amazon Web Services) S3 (Simple Storage Service) as my primary storage solution. All the ingested data are directly stored in an S3 bucket. I created a catalog in the Unity catalog in Databricks which points to my S3 bucket via external location. I have followed the [medallion architecture](#) to store my data. The bronze and silver layer data is stored in the S3 bucket, but the curated layer is stored in the Azure Data Lake Storage, which is the default storage available in the Databricks workspace. I use the VACUUM operation in Delta Lake to remove obsolete data files from my Delta tables, thereby reducing storage usage.

Orchestration

Running a databricks job every two minutes to ingest the recent changes is impossible as I hit the daily compute limit very fast. So, I created a Python DAG that is scheduled and executed using Apache Airflow, an open-source workflow orchestration platform. I deployed Airflow on an AWS Elastic Compute Cloud (EC2) instance (t3.micro, to save credit), which serves as my server. Airflow provides a web-based user interface that I use to monitor the DAG execution and its status.

Ingestion of station metadata and full changes are simple Databricks jobs.

Transformation

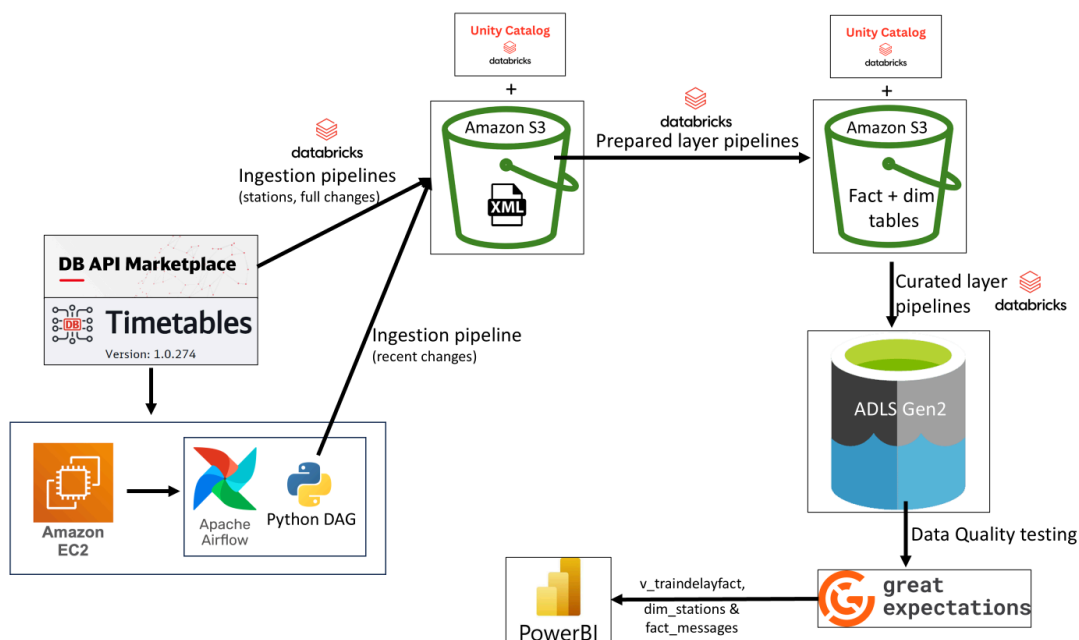
API calls create XML files, which are read using Spark's DataFrameReader. The necessary fields are extracted and some new fields are created. An event (EventId) is the primary key in my fact tables and corresponds to the scenario of a train arriving/ departing a particular station. DB can update an event multiple times (a train can get delayed further, or get cancelled after it is delayed, for eg.). So I used SCD2 to keep track of the active rows of each

event. In my prepared layer, I have two fact tables, one that keeps track of changed platforms/ changed arrival (departure) time and cancellations and another for tracking the messages (delay/ cancellation reasons) broadcasted by DB. I have one dimension table for station metadata. These delta tables are directly written onto S3. My curated layer consists of one view where I calculate the train delays explicitly.

Serving

I have created a PowerBI dashboard, which shows some important KPIs like cancellation rate, on-time performance, moderate delays etc. The PowerBI dashboard can be viewed [here](#). There exists a scope to use ML algorithms to find relationships between train delay and cause of delay.

Architecture Diagram



Challenges & Lessons learned

The hardest part of this project was data modeling. The data ingested in the form of XML files had a complex structure and parsing them in the right and meaningful way took a lot of effort compared to the other steps. Over the duration of the project, I had to revise the schema of fact tables multiple times.

Incorrect behaviour of API endpoint

Sometimes, the station metadata endpoint works incorrectly and only returns a handful of stations. So I added a small data quality check, where if the number of active central stations in the `dim_stations` is below 150, I time travel to the previous version of this table (this feature is available since this is a delta table) and restore it.

Spontaneously planned new events

I depend on the planned timetable to calculate the delay of a train, but sometimes, certain events are unavailable in the planned timetable and DB directly sends them through the recent changes endpoint. This required a careful workaround to make sure the curated layer showed the correct result.

Unreliable performance of Airflow

Considering the budget, I had to choose a `t3.micro` instance in the EC2 with 1GB of RAM, due to which the Airflow server frequently stopped as the OS ran into memory issues. I need to manually restart Airflow in this scenario. When this happens, the recent changes ingestion stops and I lose this data. This leads to incorrect/ flawed KPIs unless the corresponding events are also removed from the planned time table data.

Data Quality testing

I have used Great Expectations, a Python framework to implement data quality testing. The testing notebook validates the gold layer after pipelines are run. I defined 9 expectations, which examines the completeness in the primary keys, distribution of categorical variables, non-null constraints on important columns and plausible values in the delay. In the case, where any of the expectation fails, an error message is raised and the fact tables need to

be reverted to their previous version manually. Scope exists to perform data quality testing even before the prepared layer pipelines are run, so that the above manual step can be avoided.